

# Reviewer Pack — Minimal Geometric Entropy and Circuit Depth Inequality

Entry ff70ea398c27 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 03:03:15 UTC

## Minimal Geometric Entropy and Circuit Depth Inequality

**Entry ID:** ff70ea398c27 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 03:03:15 UTC

### 1. Conjecture

**Field A** (mathematical branch): Noncommutative Geometry **Field B** (complexity object): Boolean Circuit Complexity

#### Statement:

For every satisfiability problem instance  $\varphi$  with  $n$  variables, the geometric entropy of the corresponding quantum system  $Q(\varphi)$  is lower bounded by a function  $\varepsilon(\varphi)$  such that  $\varepsilon(\varphi) = \Omega(n^{1/3})$ . Moreover, for all instances  $\varphi$ , the circuit depth  $D(\varphi)$  satisfies  $D(\varphi) \leq C\varepsilon(\varphi)$ , where  $C$  is a constant.

#### Rationale (proposer's reasoning):

Noncommutative geometry offers a novel framework to study complex systems. The geometric entropy of quantum systems has been shown to relate to complexity measures in other contexts. By applying this concept to the satisfiability problem, we may uncover new structural insights into computational complexity.

**Taxonomy category:** Quantum\_Complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 563a54298de8fec0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For each satisfiability instance with  $n$  variables, if the geometric entropy  $\varepsilon(\varphi)$  is lower bounded by  $n^{1/3}$  with at least 30 seeds and the circuit depth  $D(\varphi)$  is less than or equal to a constant  $C$  times  $\varepsilon(\varphi)$ , then the conjecture is supported.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 0.90       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 0.90       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - "geometric entropy" AND "Boolean circuit complexity" - "quantum system  $Q(\phi)$ " AND "circuit depth  $D(\phi)$ " - "Noncommutative Geometry" AND " $n^{(1/3)}$  lower bound"

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=240.3s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def geometric_entropy(instance):
        n = len(instance)
        count_ones = instance.count(1)
        count_zeros = n - count_ones
        if count_ones == 0 or count_zeros == 0:
            return 0
        p_one = count_ones / n
        p_zero = count_zeros / n
        entropy = -p_one * math.log2(p_one) - p_zero * math.log2(p_zero)
        return entropy

    def circuit_depth(instance):
        # Simplified DPLL solver to estimate circuit depth
        n = len(instance)
        if n == 1:
            return 1
        return 2 + circuit_depth(instance[:n//2]) + circuit_depth(instance[n//2:])

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        instance = generate_instance(n)
        entropy = geometric_entropy(instance)
        depth = circuit_depth(instance)
        if entropy == 0 or depth == 0:
```

```

        continue
    results.append({
        "n": n,
        "geometric_entropy": entropy,
        "circuit_depth": depth
    })

if not results:
    return {
        "metric_name": "geometric_entropy",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 40,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n_max = max(r["n"] for r in results)
mean_entropy = sum(r["geometric_entropy"] for r in results) / len(results)
mean_depth = sum(r["circuit_depth"] for r in results) / len(results)

conjecture_holds = all(entropy >= n**(1/3) and depth <= 2 * entropy for n, entropy, depth in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "geometric_entropy",
    "metric_value": mean_entropy,
    "instances_tested": len(results),
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out before producing data, which means the pre-registered support conditions could not be unambiguously met. | next: NONE

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 14602        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 10156        |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9089         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 13629        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 30616        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8854         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8450         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10292        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 12342        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118030 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/ff70ea398c27.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ff70ea398c27.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/ff70ea398c27.tar.gz` (if generated)